

犁耙入地深度检测算法开发文档

开发总纲 / Implementation Master Plan

适用场景：标定板可见，且标定板与犁耙相对位置固定的入地深度实时检测任务

项目目标	基于标定板位姿和地面参考平面，实时估计犁耙标定点到地面的距离，并输出耕作深度。
核心输入	ZED RGB 图像、CameraInfo、点云 / 深度数据。
核心输出	标定板位姿、犁耙标定点三维位置、地面平面参数、入地深度、RViz 可视化结果。
开发原则	按模块分阶段实现；每一步先打通、测试、验收，再进入下一步，避免多模块同时调试。

开发总流程：数据链路 → 标定板位姿 → 犁耙点变换 → 地面平面 → 深度计算 → RViz 可视化

1. 项目目标与范围

本项目面向犁耙入地深度实时检测任务，在“标定板可见、且标定板与犁耙相对位置固定”的条件下，建立一套可工程落地的视觉—几何联合测量方法。

- 检测标定板并估计其相对于相机的位姿；
- 利用标定板与犁耙标定点之间的固定外参，计算犁耙测量点三维位置；
- 将地面定义为参考平面，并通过点云拟合求得平面参数；
- 计算犁耙标定点到参考平面的距离，作为耕作深度；
- 在 RViz 中发布深度结果与调试可视化。

2. 开发原则

- 先打通 ROS2 数据链路，再做算法模块，不在数据未通时调算法；
- 先实现单一功能，再做模块集成，每一步都必须有明确的测试和验收标准；
- 第一版追求“稳定跑通”，后续再逐步提升精度、鲁棒性和实时性；
- 第一版统一在相机坐标系下完成计算，不优先引入 map / odom 全局坐标系；
- 每一阶段都保留调试输出和 RViz 可视化，便于快速定位问题。

3. 算法总体链路

阶段	输入	输出
标定板检测与位姿估计	RGB 图像 + CameraInfo	T_C_B
犁耙标定点位置解算	T_C_B + p_B	p_C
地面 ROI 提取与平面拟合	点云 / 深度	$\pi = [a,b,c,d]$
入地深度计算	$p_C + \pi$	signed_dist, depth
RViz 可视化	位姿 / 点 / 平面 / 深度	Marker / 文本 / 线段

4. 坐标系与核心变量

- C：相机坐标系，作为第一版算法的统一参考坐标系；
- B：标定板坐标系，通过视觉检测实时求得 T_C_B；
- P：犁耙标定点，定义在犁耙上的固定深度测量点。

核心变量定义如下：

- T_C_B：标定板相对于相机的位姿变换；
- $p_B = [x_B, y_B, z_B]^T$ ：犁耙标定点在板坐标系下的位置参数；
- $p_C = T_C_B \cdot p_B$ ：犁耙标定点在相机坐标系下的三维位置；
- $\pi = [a,b,c,d]$ ：地面参考平面参数，满足 $ax + by + cz + d = 0$ 。

5. 代码总体框架

建议将项目组织为一个独立的 ROS2 包，形成“主节点 + 功能模块 + 配置文件 + 启动文件”的结构。

```
tillage_depth_estimator/  
├── package.xml  
├── setup.py  
├── launch/  
│   └── tillage_depth.launch.py  
├── config/  
│   └── tillage_depth.yaml  
├── tillage_depth_estimator/  
│   ├── tillage_depth_node.py  
│   ├── board_detector.py  
│   ├── tool_transform.py  
│   ├── ground_plane.py  
│   ├── depth_estimator.py  
│   ├── visualization.py  
│   └── utils.py  
└── test/
```

6. 模块职责划分

模块	文件建议	职责
主节点	tillage_depth_node.py	订阅图像、内参、点云；调度各模块；发布最终结果
标定板检测	board_detector.py	检测标定板并估计 T_{C_B}
犁耙点变换	tool_transform.py	根据固定外参 p_B 求解 p_C
地面平面拟合	ground_plane.py	提取 ground ROI 并拟合平面
深度计算	depth_estimator.py	计算点到平面距离并输出深度
可视化	visualization.py	发布 Marker、线段、文本等 RViz 调试信息

7. 分阶段开发步骤与验收要求

Step 1: 搭建 ROS2 节点骨架与数据链路

目标：先不做算法，只完成节点启动、订阅 ZED 图像 / CameraInfo / 点云，并输出基本状态信息。

- 要解决的问题：数据是否能稳定进入算法节点。
- 实现内容：Node 类、订阅器、图像回调、CameraInfo 回调、点云回调、状态标志位。
- 测试方法：观察是否能持续收到图像、内参和点云；检查回调频率。
- 验收标准：节点稳定运行，三类数据均可接收，无明显阻塞和异常退出。

Step 2: 实现标定板检测与位姿估计

目标：从 RGB 图像中检测标定板，并输出其相对于相机的位姿 T_{C_B} 。

- 要解决的问题：如何稳定地从图像中得到标定板 6D 位姿。
- 实现内容：Aruco / ChArUco 检测、角点解析、solvePnP 或 marker pose 估计。
- 测试方法：静态放置标定板，观察检测率与位姿连续性。
- 验收标准：板可见时检测稳定，位姿无明显跳变，失检时能正确给出失败状态。

Step 3: 实现标定板到犁耙标定点的固定外参变换

目标：根据 T_{C_B} 和固定参数 p_B ，计算犁耙标定点 p_C 。

- 要解决的问题：如何从“板的位置”稳定推算“犁耙测量点的位置”。
- 实现内容：坐标变换函数 `transform_tool_point( $T_{C\_B}$ ,  $p_B$ )`。
- 测试方法：整体移动板与犁耙，观察 p_C 是否同步变化。
- 验收标准：位置连续，坐标轴方向正确，外参与实际几何关系一致。

Step 4: 实现地面 ROI 提取与平面拟合

目标：从点云中提取地面区域并拟合参考平面 π 。

- 要解决的问题：如何定义可靠的地面基准。
- 实现内容：ROI 裁剪、无效点过滤、RANSAC 平面拟合。
- 测试方法：静态环境下观察平面法向和拟合内点。
- 验收标准：平面参数稳定，法向方向合理，拟合结果贴近真实地面。

Step 5: 实现深度计算

目标：计算犁耙标定点到参考平面的有符号距离，并输出入地深度。

- 要解决的问题：如何将几何关系转成可解释的深度值。
- 实现内容：点到平面距离公式、`signed_dist` 与 `depth` 输出。
- 测试方法：手动抬高/压低犁耙，观察深度值变化。
- 验收标准：深度变化方向正确，数值连续稳定，与人工量测趋势一致。

Step 6: 实现 RViz 可视化

目标：可视化板位姿、犁耙点、参考平面法向、垂线和深度文本。

- 要解决的问题：如何直观验证算法结果是否合理。
- 实现内容：Sphere / Arrow / Line / Text Marker 发布。
- 测试方法：观察 Marker 是否与真实位置一致，文本是否实时更新。
- 验收标准：RViz 显示正确、坐标系一致、深度结果可读。

Step 7: 整合、参数化与鲁棒性优化

目标：在前面各模块通过后，再统一整合，并增加参数化与异常保护。

- 要解决的问题：失检、平面拟合失败、抖动和参数调整困难。
- 实现内容：YAML 参数、失检保护、平滑滤波、调试日志分级。
- 测试方法：长时间运行、弱纹理、轻微遮挡等工况测试。
- 验收标准：系统稳定、参数可调、失检时不会引发整链崩溃。

8. 当前开发起点建议

当前阶段建议只做两件事：

- 建立 ROS2 包和基础文件结构；
- 完整实现 Step 1：节点骨架 + ZED 三类数据订阅。

只有在 Step 1 完全通过后，再进入 Step 2 做标定板检测。这样可以把“通信 / 数据链路问题”和“算法问题”严格分开。

9. 参数文件建议框架

```
board:
  method: aruco
  marker_length: 0.05
  dictionary: DICT_4X4_50

tool:
  point_in_board: [0.0, 0.0, 0.0]

ground:
  roi_x_min: 0.3
  roi_x_max: 1.5
  roi_y_min: -0.5
  roi_y_max: 0.5
  roi_z_min: -0.2
  roi_z_max: 1.0
  ransac_threshold: 0.01
  min_inliers: 300

depth:
  clamp_negative_to_zero: true
  smooth_alpha: 0.3

visualization:
  frame_id: zed_left_camera_frame
```

10. 建议预留的核心函数接口

```
class TillageDepthNode(Node):
    def image_callback(self, msg): ...
    def camera_info_callback(self, msg): ...
    def pointcloud_callback(self, msg): ...
    def process_once(self): ...

detect_board_pose(image, camera_info)
transform_tool_point(T_C_B, p_B)
extract_ground_roi(point_cloud, cfg)
fit_ground_plane(points)
compute_depth(point, plane)
publish_markers(...)
```

11. 结论

本开发文档给出的并不是一次性完成的“大而全实现方案”，而是一套适合工程开发的分步推进框架。

建议严格按照“Step 1 → Step 2 → Step 3 → Step 4 → Step 5 → Step 6 → Step 7”的顺序实施。

当前最合理的下一步，是先进入 Step 1，完成 ROS2 包建立、节点骨架编写和 ZED 数据订阅打通。